

2주차 예습과제

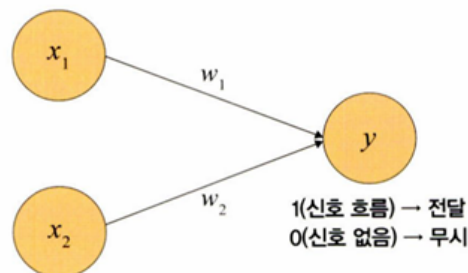
📖 딥러닝 파이토치 교과서 4장

4장. 딥러닝 시작

4.1 인공 신경망의 한계와 딥러닝 출현

- 퍼셉트론
 - 오늘날 인공 신경망에서 이용하는 구조(입력층, 출력층, 가중치로 구성된 구조)
 - 프랭크 로젠블라트가 1957년에 고안
 - 다수의 신호(흐름이 있는)를 입력으로 받아 하나의 신호를 출력, 이 신호를 입력으로 받아 '흐른다/안 흐른다(1 또는 0)'는 정보를 앞으로 전달하는 원리로 작동

▼ 그림 4-1 퍼셉트론 원리



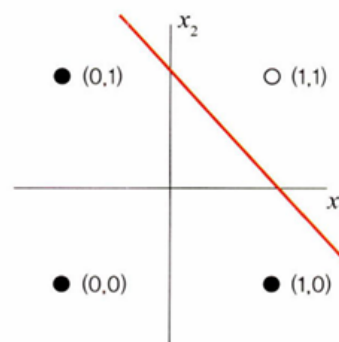
AND 게이트

- 모든 입력이 '1'일 때 작동 = 입력 중 어떤 하나라도 '0'을 갖는다면 작동 멈춤

▼ 표 4-1 AND 게이트

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

▼ 그림 4-2 AND 게이트



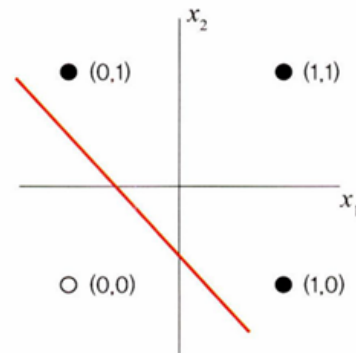
OR 게이트

- 입력에서 둘 중 하나만 '1'이거나 둘 다 '1'일 때 작동

▼ 표 4-2 OR 게이트

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

▼ 그림 4-3 OR 게이트



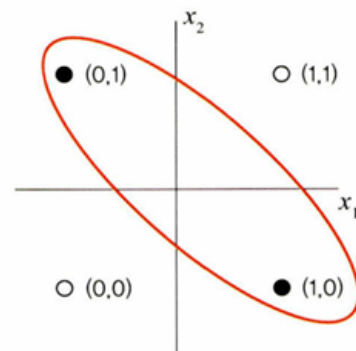
XOR 게이트

- 배타적 논리합, 입력 두 개 중 한 개만 '1'일 때 작동하는 논리 연산
- 데이터가 비선형적으로 분리되기 때문에 제대로 된 분류 어려움

▼ 표 4-3 XOR 게이트

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

▼ 그림 4-4 XOR 게이트



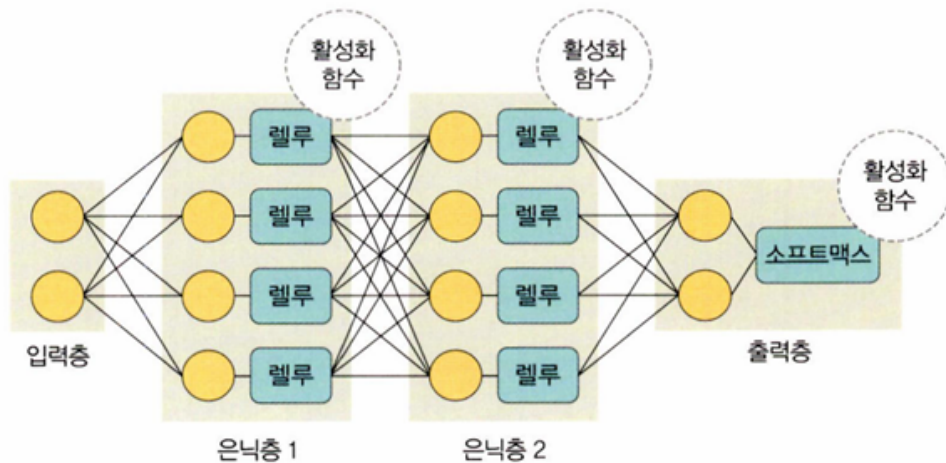
- 즉, 단층 퍼셉트론에서는 AND, OR 연산은 학습 가능하지만 XOR은 학습 불가능
- 이를 극복하기 위해 다층 퍼셉트론(multi-layer perceptron)을 고안
 - 입력층과 출력층 사이 하나 이상의 중간층(은닉층)을 두어 비선형적으로 분리되는 데이터에 대해서도 학습이 가능하도록 함
 - 심층 신경망(Deep Neural Network, DNN): 입력층과 출력층 사이에 은닉층이 여러 개 있는 신경망, 다른 이름으로 딥러닝

4.2 딥러닝 구조

4.2.1 딥러닝 용어

- 딥러닝은 입력층, 출력층과 두 개 이상의 은닉층으로 구성되어 있음
- 입력 신호를 전달하기 위해 다양한 함수 사용

▼ 그림 4-5 딥러닝 구조

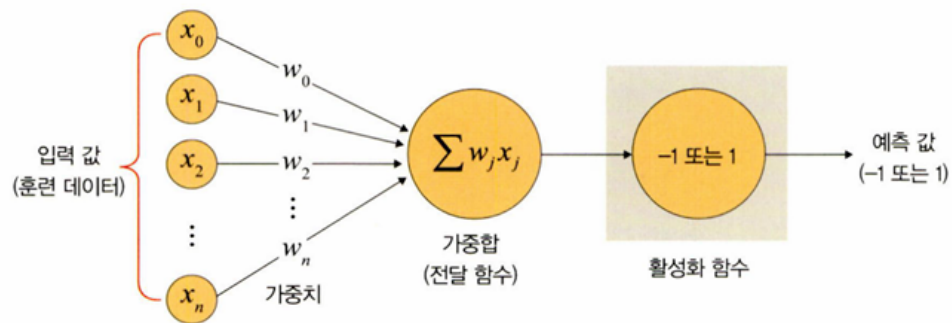


구분	구성 요소	설명
층	입력층(input layer)	데이터를 받아들이는 층
	은닉층(hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
	출력층(output layer)	신경망의 최종 결과값이 포함된 층
가중치(weight)		노드와 노드 간 연결 강도
바이어스(bias)		가중합에 더해 주는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할을 함
가중합(weighted sum), 전달 함수		가중치와 신호의 곱을 합한 것
함수	활성화 함수 (activation function)	신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
	손실 함수 (loss function)	가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

가중치

- 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소
 - ex) w_1 값이 0 혹은 0과 가까운 값이라면 x_1 이 아무리 큰 값이어도 $x_1 \times w_1$ 은 0이거나 0에 가까운 값이 됨

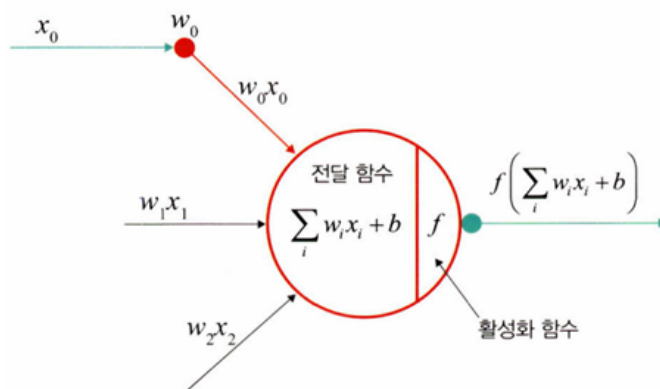
♥ 그림 4-6 가중치



가중합 또는 전달 함수

- 각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달될 때, 이 값들을 모두 더한 합
- 노드의 가중합이 계산되면 이 가중합을 활성화 함수로 보내기 때문에 전달 함수 (transfer function)라고도 함

♥ 그림 4-7 전달 함수



- 가중합 공식: $\sum_i w_i x_i + b$ (w :가중치, b : 바이어스)

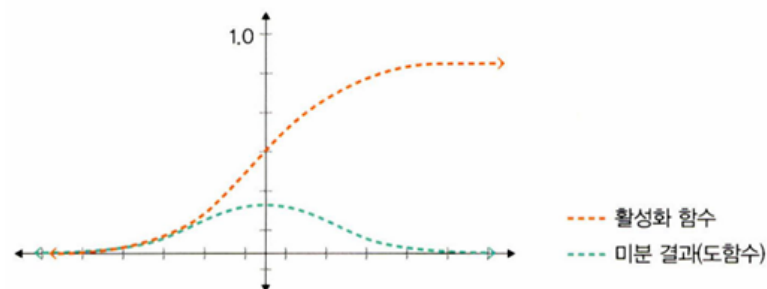
활성화 함수

- 전달 함수에서 전달받은 값을 출력 할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수
- 시그모이드(sigmoid), 하이퍼볼릭 탄젠트(hyperbolic tangent), 렐루(ReLU) 함수 등

시그모이드 함수

- 선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형
- 주로 로지스틱 회귀 같은 분류 문제를 확률적으로 표현하는 데 사용
- 모델의 깊이가 깊어지면 기울기가 사라지는 '기울기 소멸 문제(vanishing gradient problem)'가 발생하여 딥러닝 모델에서는 잘 사용하지 않음
- 시그모이드 함수 수식: $f(x) = \frac{1}{1+e^{-x}}$

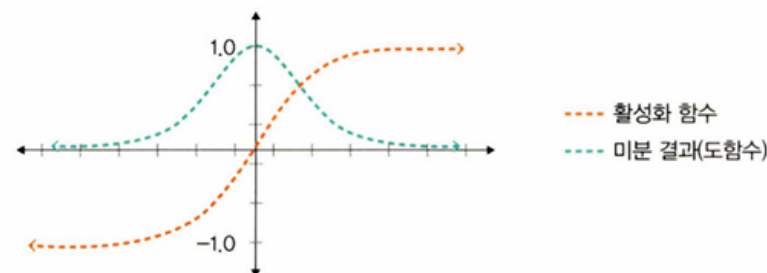
▼ 그림 4-8 시그모이드 활성화 함수와 미분 결과



하이퍼볼릭 탄젠트 함수

- 선형 함수의 결과를 -1~1 사이에서 비선형 형태로 변형
- 시그모이드에서 결괏값의 평균이 0이 아닌 양수로 편향된 문제를 해결하는 데 사용했지만 기울기 소멸 문제가 여전히 발생

▼ 그림 4-9 하이퍼볼릭 탄젠트 활성화 함수와 미분 결과

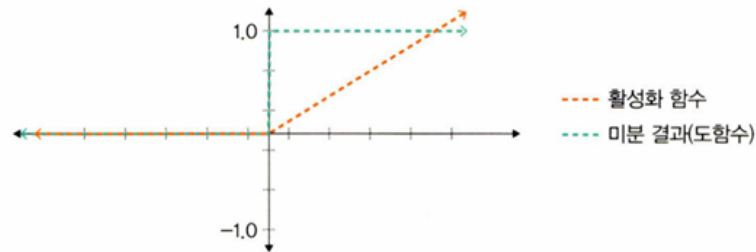


렐루 함수

- 입력(x)이 음수일 때는 0을 출력하고, 양수일 때는 x를 출력
- 경사 하강법(gradient descent)에 영향을 주지 않아 학습 속도가 빠르고, 기울기 소멸 문제가 발생하지 않는 장점
- 일반적으로 은닉층에서 사용되고, 하이퍼볼릭 탄젠트 함수 대비 학습 속도가 6배 빠름

- 음수 값을 입력받으면 항상 0을 출력하기 때문에 학습 능력이 감소하는 문제 → 리키 렐루 함수

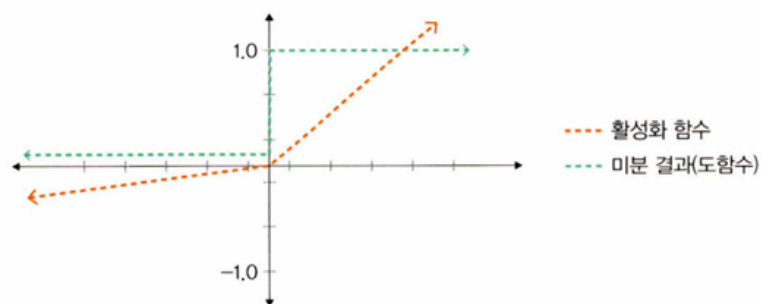
▼ 그림 4-10 렐루 활성화 함수와 미분 결과



리키 렐루 함수

- 입력 값이 음수이면 0이 아닌 0.001처럼 매우 작은 수를 반환
- 입력 값이 수렴하는 구간이 제거되어 렐루 함수를 사용할 때 생기는 문제를 해결

▼ 그림 4-11 리키 렐루 활성화 함수와 미분 결과



소프트맥스 함수

- 입력 값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 함
- 딥러닝에서 출력 노드의 활성화 함수로 많이 사용됨
- 소프트맥스 함수 수식: $y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$
 - $\exp(x)$ 는 지수 함수
 - n 은 출력층의 뉴런 개수
 - y_k 는 그 중 k 번째 출력
 - 소프트맥스 함수의 분자는 입력 신호 a_k 의 지수 함수, 분모는 모든 입력 신호의 지수 함수 합

```
# 렐루 함수와 소프트맥스 함수 구현 코드(코랩에 실습 옮기기)
class Net(torch.nn.Module):
    def __init__(self, n_feature, n_hidden, n_output):
        super(Net, self).__init__()
        self.hidden = torch.nn.Linear(n_feature, n_hidden) #은닉층
        self.relu = torch.nn.ReLU(inplace=True)
        self.out = torch.nn.Linear(n_hidden, n_output) #출력층
        self.softmax = torch.nn.Softmax(dim=n_output)
    def forward(self, x):
        x = self.hidden(x)
        x = self.relu(x) #은닉층을 위한 렐루 활성화 함수
        x = self.out(x)
        x = self.softmax(x) #출력층을 위한 소프트맥스 활성화 함수
        return x
```

손실 함수

- 경사 하강법: 학습률(η , learning rate)과 손실 함수의 순간 기울기를 이용하여 가중치를 업데이트 하는 방법
- 미분의 기울기를 이용하여 오차를 비교하고 최소화하는 방향으로 이동시키는 방법
- 이때 오차를 구하는 방법이 손실 함수로, 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표
- 값이 클수록 많이 틀렸다는 의미, 값이 '0'에 가까울수록 완벽하게 추정할 수 있다는 의미
- 대표적인 손실 함수로 평균 제곱 오차(Mean Squared Error, MSE)와 크로스 엔트로피 오차(Cross Entropy Error, CEE)가 있음

평균 제곱 오차(MSE)

- 실제 값과 예측 값의 차이(error)를 제곱하여 평균을 낸 것
- 실제 값과 예측 값의 차이가 클수록 커진다 → 값이 작을수록 예측력이 좋다
- 회귀에서 손실 함수로 주로 사용
- $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
 - $\hat{y}_i$: 신경망의 출력(신경망이 추정한 값)
 - y_i : 정답 레이블

- i : 데이터의 차원 개수

```
#파이토치에서 평균 제곱 오차 사용법
import torch

loss_fn = torch.nn.MSELoss(reduction='sum')
y_pred = model(x)
loss = loss_fn(y_pred, y)
```

크로스 엔트로피 오차

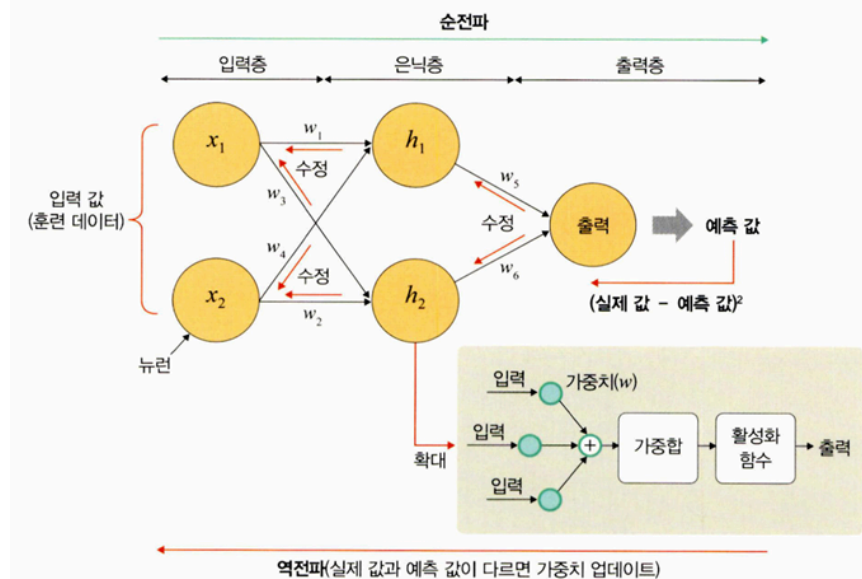
- 분류 문제에서 원-핫 인코딩(one-hot encoding)했을 때만 사용할 수 있는 오차 계산법
- 일반적으로 분류 문제에서는 데이터의 출력을 0과 1로 구분하기 위해 시그모이드 함수를 사용하는데, 이때 시그모이드 함수에 포함된 자연 상수 e 때문에 평균 제곱 오차를 적용하면 매끄럽지 못한 그래프가 출력되기 때문에 크로스 엔트로피 손실 함수를 사용한다
- 크로스 엔트로피 손실 함수를 적용할 경우 경사 하강법 과정에서 학습이 지역 최소점에 머물 수 있기 때문에 이를 방지하기 위해 자연 상수 e 에 반대되는 자연 로그를 모델의 출력 값에 취한다
- $CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$
 - $\hat{y}_i$: 신경망의 출력(신경망이 추정한 값)
 - y_i : 정답 레이블
 - i : 데이터의 차원 개수

```
#파이토치에서 크로스 엔트로피 오차 사용법
loss = nn.CrossEntropyLoss()
input = torch.randn(5, 6, requires_grad=True) #torch.randn은 평균이 0이고
표준편차가 1인 가우시안 정규분포를 이용하여 숫자를 생성
target = torch.empty(3, dtype=torch.long).random_(5) #torch.empty는 dtype
torch.float32의 랜덤한 값으로 채워진 텐서를 반환
output.backward()
```

4.2.2 딥러닝 학습

- 크게 순전파와 역전파라는 두 단계로 진행됨

♥ 그림 4-12 순전파와 역전파



• 순전파(feedforward)

- 네트워크에 훈련 데이터가 들어올 때 발생
- 데이터를 기반으로 예측 값을 계산하기 위해 전체 신경망을 교차해 지나감
- 모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 변환(가중합 및 활성화 함수)을 적용하여 다음 층(은닉층)의 뉴런으로 전송하는 방식
- 네트워크를 통해 입력 데이터를 전달, 데이터가 모든 층을 통과하고 모든 뉴런이 계산을 완료하면 그 예측 값이 최종 층(출력층)에 도달
- 손실 함수로 네트워크의 예측 값과 실제 값의 차이(손실, 오차)를 추정
- 손실 함수 비용은 '0'이 이상적, 비용이 0에 가깝도록 모델이 훈련을 반복하며 가중치를 조정

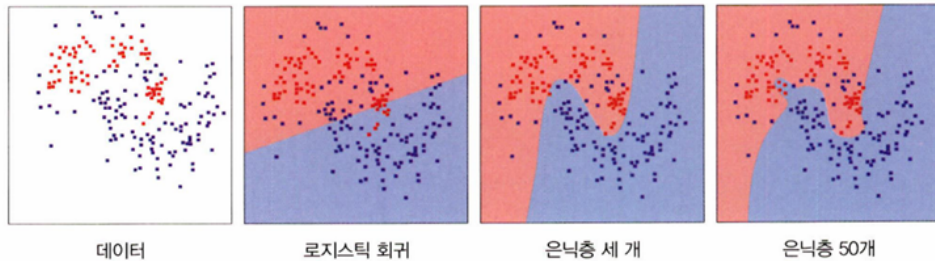
• 역전파(backpropagation)

- 손실(오차)이 계산되면 그 정보는 역으로 전파(출력층 → 은닉층 → 입력층)됨
- 출력층에서 시작된 손실 비용은 은닉층의 모든 뉴런으로 전파되지만, 은닉층의 뉴런은 각 뉴런이 원래 출력에 기여한 상대적 기여도에 따라(가중치에 따라) 값이 달라짐
- 예측 값과 실제 값 차이를 각 뉴런의 가중치로 미분한 후 기존 가중치 값에서 빼는 이 과정을 출력층 → 은닉층 → 입력층 순서로 모든 뉴런에 대해 진행, 계산된 각 뉴런 결과를 다시 순전파의 가중치 값으로 사용

4.2.3 딥러닝의 문제점과 해결 방안

- 딥러닝의 핵심은 활성화 함수가 적용된 여러 은닉층을 결합하여 비선형 영역을 표현하는 것
- 활성화 함수가 적용된 은닉층 개수가 많을수록 데이터 분류가 잘 되지만 문제가 발생

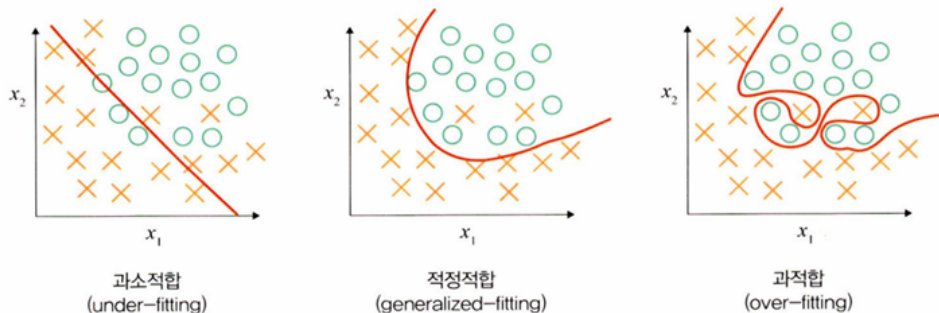
▼ 그림 4-13 은닉층이 분류에 미치는 영향



과적합 문제 발생

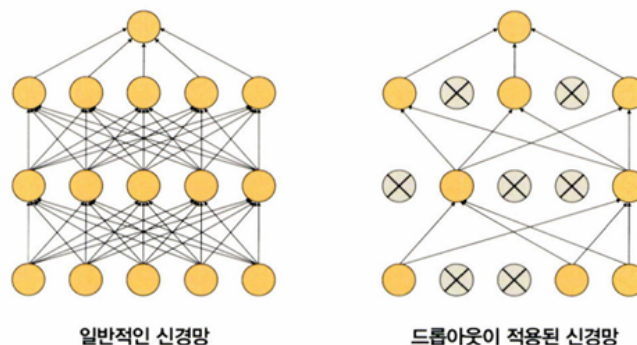
- 과적합(over-fitting)은 훈련 데이터를 과하게 학습해서 발생
- 훈련 데이터에 대해 과하게 학습하여 실제 데이터에 대한 오차가 증가하는 현상

▼ 그림 4-14 과적합



- 드롭아웃(dropout)
 - 신경망 모델이 과적합되는 것을 피하기 위한 방법으로, 학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴

▼ 그림 4-15 일반적인 신경망과 드롭아웃이 적용된 신경망



드롭아웃 구현 예시 코드

```
class DropoutModel(torch.nn.Module):
    def __init__(self):
        super(DropoutModel, self).__init__()
        self.layer1 = torch.nn.Linear(784, 1200)
        self.dropout1 = torch.nn.Dropout(0.5) #50%의 노드를 무작위로 선택하여
        사용하지 않겠다는 의미
        self.layer2 = torch.nn.Linear(1200, 1200)
        self.dropout2 = torch.nn.Dropout(0.5)
        self.layer3 = torch.nn.Linear(1200, 10)

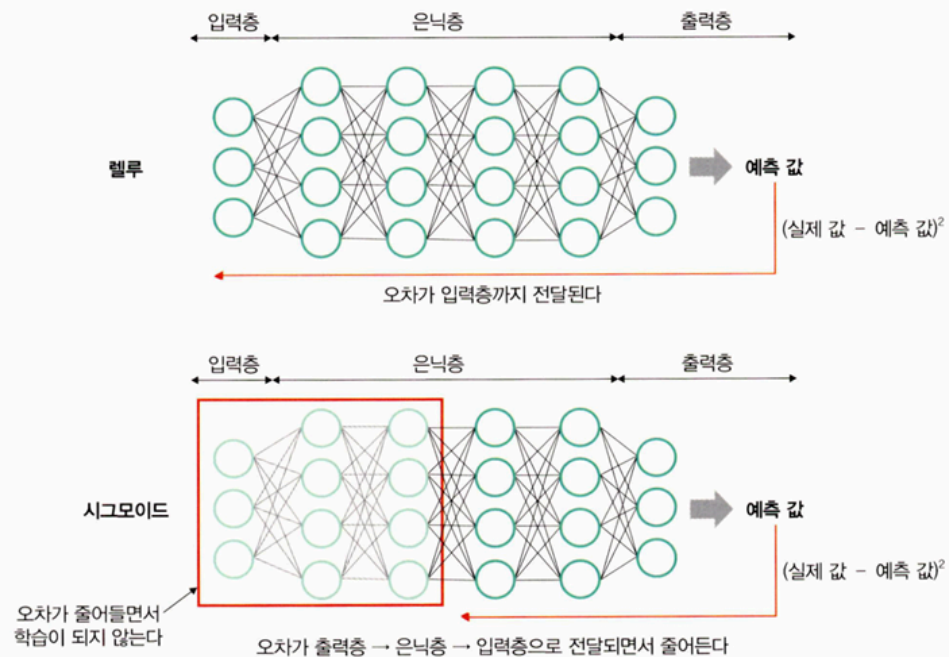
    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = self.dropout1(x)
        x = F.relu(self.layer2(x))
        x = self.dropout2(x)
        return self.layer3(x)
```

기울기 소멸 문제 발생

- 은닉층이 많은 신경망에서 주로 발생
- 출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상
- 기울기가 소멸되기 때문에 학습되는 양이 '0'에 가까워져 학습이 더디게 진행된다 오차를 더 줄이지 못하고 그 상태로 수렴

→ 렐루 활성화 함수를 사용하면 해결

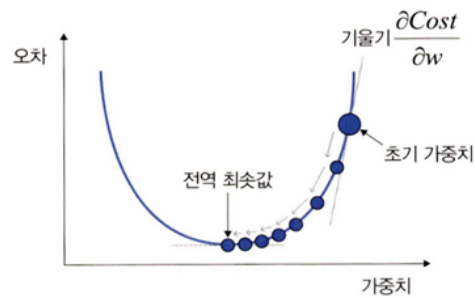
▼ 그림 4-16 기울기 소멸 문제



성능이 나빠지는 문제 발생

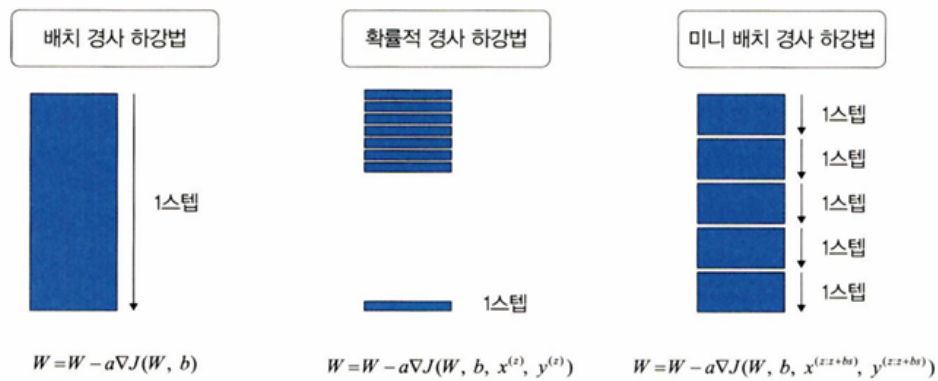
- 경사 하강법에서 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복하는데, 이때 성능이 나빠지는 문제가 발생

▼ 그림 4-17 경사 하강법



- 확률적 경사 하강법, 미니 배치 경사 하강법을 사용해 개선

▼ 그림 4-18 경사 하강법의 유형



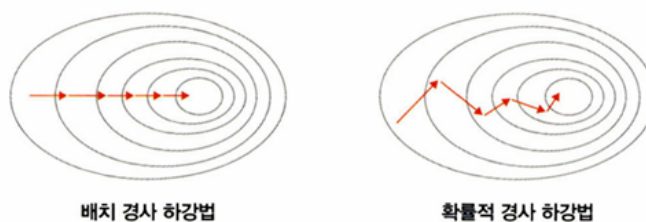
• 배치 경사 하강법(Batch Gradient Descent, BGD)

- 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산해 모델의 파라미터를 업데이트
- 즉, 전체 훈련 데이터셋에 대해 가중치를 편미분하는 방법
- $W = W - a \nabla J(W, b)$
 - 손실 함수의 값을 최소화하기 위해 기울기(∇) 이용
 - a : 학습률
 - J : 손실 함수
- 한 스텝에 모든 훈련 데이터셋을 사용하므로 학습이 오래 걸리는 단점 → 확률적 경사 하강법에서 개선

• 확률적 경사 하강법(Stochastic Gradient Descent, SGD)

- 임의로 선택한 데이터에 대해 기울기를 계산하는 방법
- 적은 데이터를 사용해 빠른 계산이 가능
- 파라미터 변경 폭이 불안정하고, 때로는 배치 경사 하강법보다 정확도가 낮을 수 있음

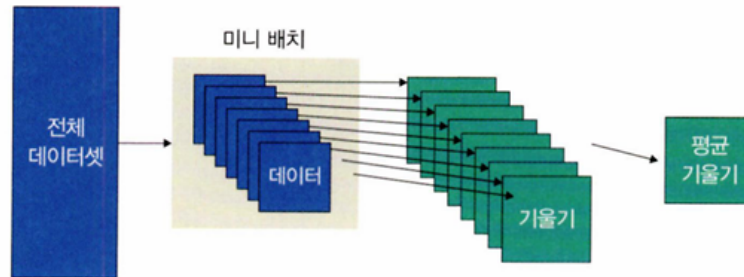
▼ 그림 4-19 배치 경사 하강법과 확률적 경사 하강법



• 미니 배치 경사 하강법(mini-batch gradient descent)

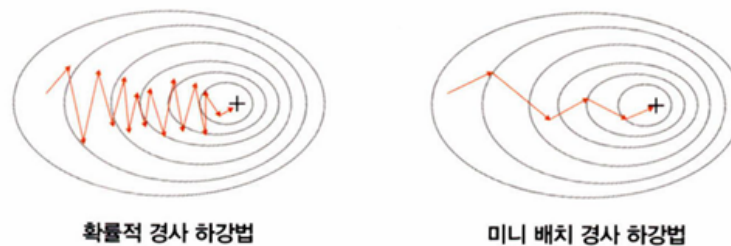
- 전체 데이터셋을 미니 배치 여러 개로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것의 평균 기울기를 이용하여 모델을 업데이트해서 학습

▼ 그림 4-20 미니 배치 경사 하강법



- 전체 데이터를 계산하는 것보다 빠르며, 확률적 경사 하강법보다 안정적 → 실제로 가장 많이 사용

▼ 그림 4-21 확률적 경사 하강법과 미니 배치 경사 하강법



미니 배치 경사 하강법 구현 코드

```
class CustomDataset(Dataset):
```

```
    def __init__(self):
```

```
        self.x_data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
        self.y_data = [[12], [18], [11]]
```

```
    def __len__(self):
```

```
        return len(self.x_data)
```

```
    def __getitem__(self, idx):
```

```
        x = torch.FloatTensor(self.x_data[idx])
```

```
        y = torch.FloatTensor(self.y_data[idx])
```

```
        return x, y
```

```
dataset = CustomDataset()
```

```
dataloader = DataLoader(
```

```
    dataset, #데이터셋
```

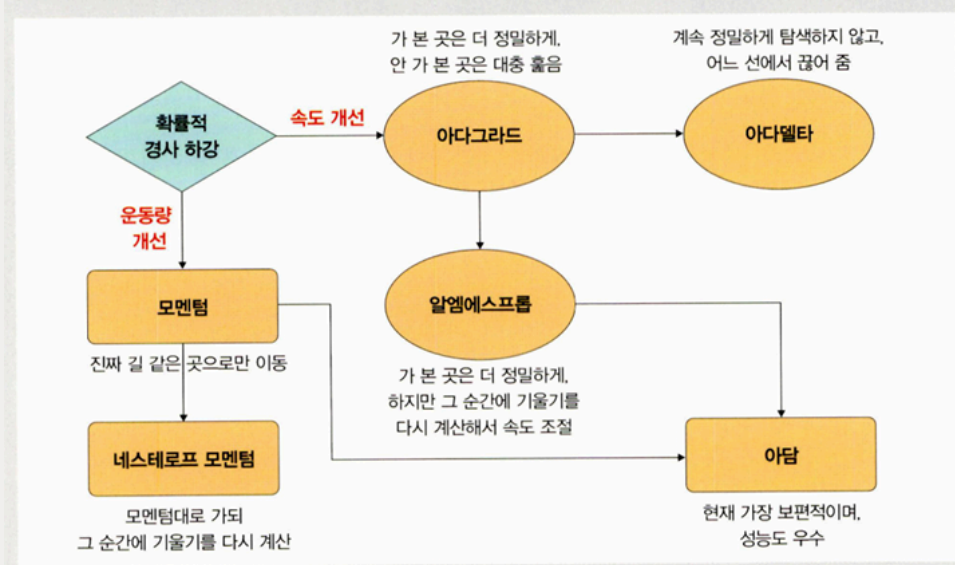
```
    batch_size=2, #미니 배치 크기로 2의 제곱수를 사용하겠다는 의미
```

shuffle=True, #데이터를 불러올 때마다 랜덤으로 섞어서 가져옴
)

▼ 옵티마이저

- 확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제를 해결하기 위해 학습 속도와 운동량을 조정하는 옵티마이저(optimizer)를 적용해 볼 수 있다

▼ 그림 4-22 옵티마이저 유형



속도를 조정하는 방법

- 아다그라드(Adagrad, Adaptive gradient)
 - 변수(가중치)의 업데이트 횟수에 따라 학습률을 조정하는 방법
 - 많이 변화하지 않는 변수들의 학습률은 크게, 많이 변화하는 변수들의 학습률은 작게 함
 - 즉, 많이 변화한 변수는 최적 값에 근접했을 것이라는 가정하에 작은 크기로 이동하면서 세밀하게 값을 조정, 적게 변화한 변수들은 학습률을 크게 하여 빠르게 오차값을 줄이고자 함

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

- 파라미터마다 다른 학습률을 주기 위해 G 함수를 추가
 - G 값은 이전 G 값의 누적(기울기 크기의 누적)

- 기울기가 크면 G 값이 커지기 때문에 $\frac{\eta}{\sqrt{G(i)+\epsilon}}$ 에서 학습률(η)는 작아진다

→ 파라미터가 많이 학습되었으면 작은 학습률로 업데이트, 파라미터 학습이 덜 되었으면 개선의 여지가 많기 때문에 높은 학습률로 업데이트

- 아다그라드는 기울기가 0에 수렴하는 문제가 있어 사용하지 않는다 → 대신 알엠에스프롭 사용

아다그라드 구현 코드

```
optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01) #
학습률 기본값은 1e-2
```

- 아다델타(Adadelta, Adaptive delta)

- 아다그라드에서 G 값이 커짐에 따라 학습이 멈추는 문제를 해결
- 아다그라드의 수식에서 학습률(η)을 D 함수(가중치의 변화량(∇) 크기를 누적한 값)로 변환했기 때문에 학습률에 대한 하이퍼파라미터가 필요하지 않음

$$w(i+1) = w(i) - \frac{\sqrt{D(i-1)+\epsilon}}{\sqrt{G(i)+\epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

$$D(i) = \gamma D(i-1) + (1-\gamma)(\nabla(w(i)))^2$$

아다델타 구현 코드

```
optimizer = torch.optim.Adadelta(model.parameters(), lr=1.0) #학
습률 기본값 1.0
```

- 알엠에스프롭(RMSProp)

- 아다그라드의 $G(i)$ 값이 무한히 커지는 것을 방지하고자 제안된 방법

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

- 아다그라드에서 학습이 안 되는 문제를 해결하기 위해 G 함수에서 γ (감마)만 추가

- 즉, G 값이 너무 크면 학습률이 작아져 학습이 안 될 수 있으므로 사용자가 γ 값을 이용하여 학습률 크기를 비율로 조정할 수 있도록 함

알엠에스프롭 구현 코드

```
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01)
```

#학습률 기본값 1e-2

운동량을 조정하는 방법

- 모멘텀(Momentum)

- 경사 하강법과 마찬가지로 매번 기울기를 구하지만, 가중치를 수정하기 전에 이전 수정 방향(+, -)을 참고하여 같은 방향으로 일정한 비율만 수정하는 방법
- 수정이 양(+)의 방향과 음(-)의 방향으로 순차적으로 일어나는 지그재그 현상이 줄어듦
- 이전 이동 값을 고려하여 일정 비율만큼 다음 값을 결정하므로 관성 효과를 얻을 수 있음
- SGD(확률적 경사 하강법)와 함께 사용

$$w(i+1) = w(i) - \eta \nabla E(w(i))$$

- 확률적 경사 하강법 수식에서 $\eta \nabla E(w(i))$ 수식을 사용해 가중치를 계산
- 기울기 크기와 반대 방향만큼 가중치를 업데이트, 즉 기울기가 크면 아래쪽 (-) 방향으로 업데이트

- SGD 모멘텀(SGD with Momentum)

- 확률적 경사 하강법에서 기울기($\eta \nabla E(w(i))$)를 속도(v)로 대체하여 사용하는 방식
- 이전 속도의 일정 부분을 반영, 즉 이전에 학습했던 속도와 현재 기울기를 반영해서 가중치를 구함

$$w(i+1) = w(i) - v(i)$$

$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i))$$

모멘텀 구현 코드

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentu
```

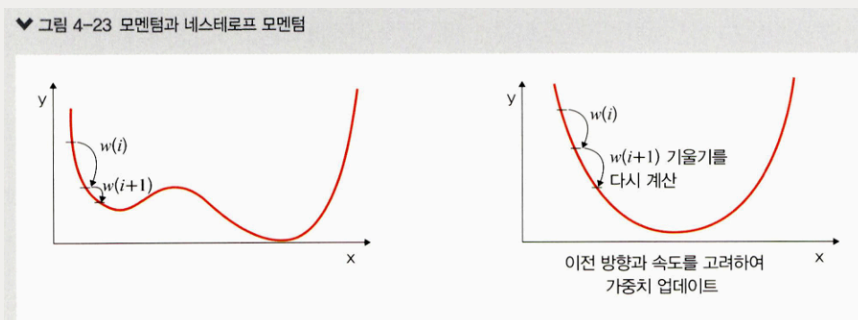
m=0.9) #momentum 값은 0.9에서 시작해 0.95, 0.99처럼 조금씩 증가시키며 사용

- 네스테로프 모멘텀(Nesterov Accelerated Gradient, NAG)

- 모멘텀 값과 기울기 값이 더해져 실제 값을 만드는 기존 모멘텀과 달리 모멘텀 값이 적용된 지점에서 기울기 값을 계산
- 네스테로프 방법은 모멘텀으로 절반 정도 이동한 후 어떤 방식으로 이동해야 하는지 다시 계산하여 결정하기 때문에 멈춰야 할 시점에서도 관성에 의해 훨씬 더 멀리 갈 수 있는 모멘텀 방법의 단점을 극복할 수 있음
→ 빠른 이동 속도는 그대로 가져가면서 멈춰야 할 적절한 시점에서 제동을 거는 데 용이함

$$w(i+1) = w(i) - v(i)$$
$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1))$$

- 속도를 구할 때 이전에 학습했던 속도와 현재 기울기에서 이전 속도를 뺀 변화량을 반영해서(더해서) 가중치를 구함



네스테로프 모멘텀 구현 코드

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9
```

```
nesterov=True) #nesterov 기
```

```
본값은 False
```

속도와 운동량에 대한 혼용 방법

- 아담(Adam, Adaptive Moment Estimation)

- 모멘텀과 알엠에스프롬의 장점을 결합한 경사하강법

- 알엠에스프롭의 특징인 기울기의 제곱을 지수 평균한 값과 모멘텀의 특징인 $v(i)$ 를 수식에 활용

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} v(i)$$

$$G(i) = \gamma_2 G(i-1) + (1 - \gamma_2) (\nabla E(w(i)))^2$$

$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

- 알엠에스프롭의 G 함수와 모멘텀의 $v(i)$ 를 사용하여 가중치를 업데이트함

아담 구현 코드

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01) # 학습률
기본값은 1e-3
```

4.2.4 딥러닝을 사용할 때 이점

특성 추출(feature extraction)

- 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 작업
- 딥러닝에서는 데이터 특성을 잘 잡아내기 위해 은닉층을 깊게 쌓는 방식으로 파라미터를 늘린 모델 구조를 통해 특성 추출 과정을 알고리즘에 통합시킴

빅데이터의 효율적 활용

- 빅데이터는 딥러닝에서 특성 추출을 알고리즘에 통합시키는 것을 가능하게 함
- 딥러닝 학습을 이용한 특성 추출은 데이터 사례가 많을수록 성능이 향상되기 때문

4.3 딥러닝 알고리즘

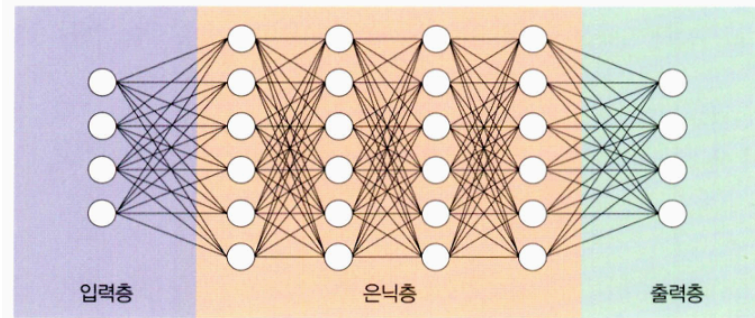
- 심층 신경망을 사용한다는 공통점이 있음
- 목적에 따라 합성곱 신경망(CNN), 순환 신경망(RNN), 제한된 볼츠만 머신(RBM), 심층 신뢰 신경망(DBN)으로 구분

4.3.1 심층 신경망

- 심층 신경망(DNN)은 입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망
- 머신 러닝에서는 비선형 분류를 하기 위해 여러 트릭을 사용했지만 심층 신경망은 다수의 은닉층을 추가했기 때문에 별도의 트릭 없이 비선형 분류가 가능

- 다수의 은닉층을 두었기 때문에 다양한 비선형적 관계를 학습할 수 있는 장점
- 학습을 위한 연산량이 많고, 기울기 소멸 문제 등이 발생할 수 있음
→ 드롭아웃, 렐루 함수, 배치 정규화 등을 적용해 해결

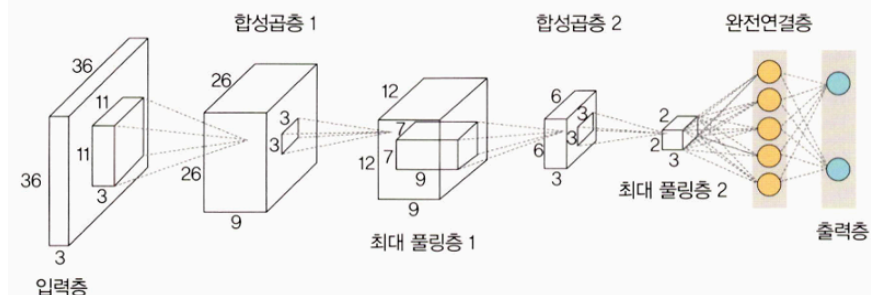
▼ 그림 4-24 심층 신경망



4.3.2 합성곱 신경망

- 합성곱 신경망(Convolutional Neural Network, CNN)은 합성곱층(convolutional layer)과 풀링층(pooling layer)을 포함하는 이미지 처리 성능이 좋은 인공신경망 알고리즘
- 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용한 신경망

▼ 그림 4-25 합성곱 신경망

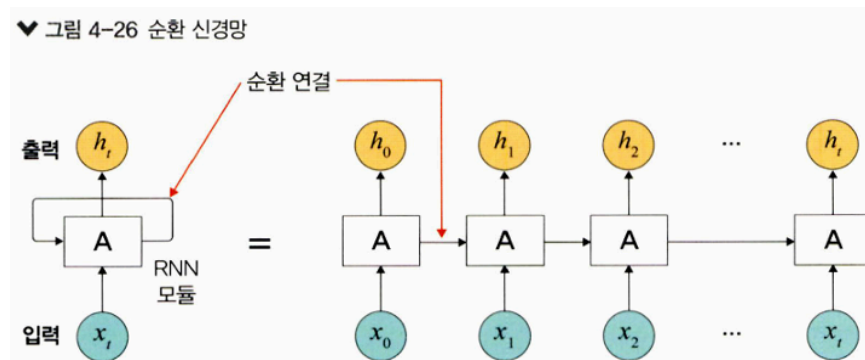


- 이미지에서 객체, 얼굴, 장면을 인식하기 위해 패턴을 찾는 데 유용
- 대표적으로 LeNet-5와 AlexNet이 있고, 층을 더 깊게 쌓은 신경망으로는 VGG, GoogLeNet, ResNet 등이 있음
- 기존 신경망과의 차별성
 - 각 층의 입출력 형상을 유지
 - 이미지의 공간 정보를 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식

- 복수 필터로 이미지의 특징을 추출하고 학습
- 추출한 이미지의 특징을 모으고 강화하는 풀링층이 있음
- 필터를 공유 파라미터로 사용하기 때문에 일반 인공 신경망과 비교하여 학습 파라미터가 매우 적음

4.3.3 순환 신경망

- 순환 신경망(Recurrent Neural Network, RNN)은 시계열 데이터(음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망
- 순환 신경망의 '순환'은 자기 자신을 참조한다는 것으로, 현재 결과가 이전 결과와 연관이 있다는 의미



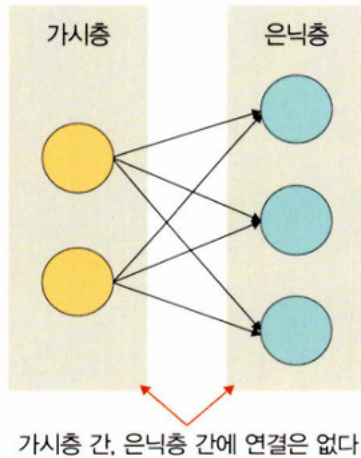
- 순환 신경망의 특징
 - 시간성(temporal property)을 가진 데이터가 많음
 - 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
 - 시간에 따라 내용이 변하므로 데이터는 동적이고, 길이가 가변적
 - 매우 긴 데이터를 처리하는 연구가 활발히 진행 중
- 기울기 소멸 문제로 학습이 제대로 되지 않는 문제가 있음
 - 메모리 개념을 도입한 LSTM(Long-Short Term Memory)이 순환 신경망에서 많이 사용됨
- 언어 모델링, 텍스트 생성, 자동 번역(기계 번역), 음성 인식, 이미지 캡션 생성 등 자연어 처리 분야와 잘 어울림

4.3.4 제한된 볼츠만 머신

- 볼츠만 머신(Boltzmann machine)은 가시층(visible layer)과 은닉층(hidden layer)으로 구성된 모델

- 가시층은 은닉층과만 연결되는데(가시층과 가시층, 은닉층과 은닉층 사이의 연결은 없음) 이것이 제한된 볼츠만 머신(Restricted Boltzmann Machine, RBM)

▼ 그림 4-27 제한된 볼츠만 머신

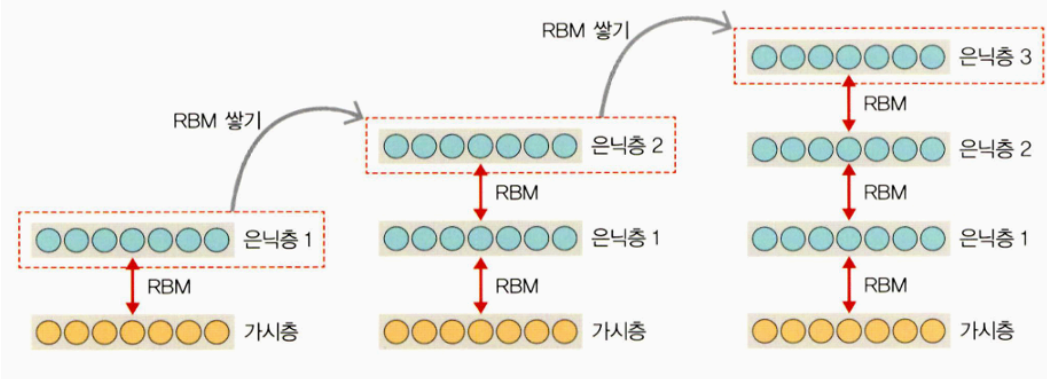


- 제한된 볼츠만 머신의 특징
 - 차원 감소, 분류, 선형 회귀 분석, 협업 필터링(collaborative filtering), 특성 값 학습(feature learning), 주제 모델링(topic modelling)에 사용함
 - 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능
 - 심층 신뢰 신경망(DBN)의 요소로 활용됨

4.3.5 심층 신뢰 신경망

- 심층 신뢰 신경망(Deep Belief Network, DBN)은 입력층과 은닉층으로 구성된 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망
- 즉, 사전 훈련된 볼츠만 머신을 층층이 쌓아 올린 구조로 레이블에 없는 데이터에 대한 비지도 학습이 가능
- 부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정을 구현할 때 사용하면 유용함
- 심층 신뢰 신경망의 학습 절차
 - 가시층과 은닉층 1에 제한된 볼츠만 머신을 사전 훈련
 - 첫 번째 층 입력 데이터와 파라미터를 고정하여 두 번째 층 제한된 볼츠만 머신을 사전 훈련
 - 원하는 층 개수만큼 제한된 볼츠만 머신을 쌓아 올려 전체 DBN을 완성

♥ 그림 4-28 심층 신뢰 신경망



- 심층 신뢰 신경망의 특징
 - 순차적으로 심층 신뢰 신경망을 학습시켜 가면서 계층적 구조를 생성
 - 비지도 학습으로 학습
 - 위로 올라갈수록 추상적 특성을 추출
 - 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용

4.4 우리는 무엇을 배워야 할까?

- 데이터를 활용하여 얻고자 하는 것에 따라 머신 러닝이나 딥러닝을 선택해서 학습하고 데이터를 훈련시키면 됨